Appendix A: Tooling and Reproduction

Appendix A: Tooling and Reproduction I

The pipeline is a two-layer system: generic infrastructure (rendering, validation, logging) shared across the template monorepo, and project-local `src/` modules that implement the meta-analysis. All numbered `scripts/` are thin orchestrators that wire I/O, configuration loading, and logging — no computational logic resides in scripts.

Reproduce the Offline Default Run I

No network, no language model required:

```
uv run python scripts/generate_fixture_corpus.py --out outp
uv run python scripts/02_meta_analysis_pipeline.py
uv run python scripts/03_build_knowledge_graph.py --max-pap
uv run python scripts/04_generate_figures.py --dpi 300
uv run python scripts/05_inject_variables.py
```

Reproduce the Live Run I

This manuscript was generated from a live retrieval run. To reproduce:

```
# Live search (all 10 engines, max 1000 per engine)
uv run python scripts/01_literature_search.py --query modal

# Analysis pipeline
uv run python scripts/02_meta_analysis_pipeline.py
uv run python scripts/03_build_knowledge_graph.py --max-pap
uv run python scripts/04_generate_figures.py --dpi 300
uv run python scripts/05_inject_variables.py
uv run python scripts/06_fulltext_assessment.py
uv run python scripts/07_literature_evaluation.py
uv run python scripts/09_export_bibliography.py
```

Re-target to Another Topic I

Edit `manuscript/config.yaml` — `project_config.search.term`, `query`, `relevance_keywords`, `subfield_keywords`, and `hypothesis_definitions` — then regenerate the seed corpus and re-run. No code changes are required; the manuscript re-targets through token injection.

Live Retrieval I

Enable engines under `project_config.search.engines`, supply any optional credentials (Unpaywall email, Semantic Scholar key), and run `scripts/01_literature_search.py`; absent engines degrade to skipped sources. The CLI supports per-engine skip flags: `--skip-arxiv`, `--skip-s2`, `--skip-openalex`, `--skip-crossref`, `--skip-pubmed`, `--skip-sovietrxiv`, `--skip-chinarxiv`, `--skip-europepmc`, `--skip-biorxiv`, `--skip-medrxiv`.

Deep Research (Offline Fixture Replay) I

This exemplar also demonstrates the shared `infrastructure.search.deep_research` capability — provider-neutral dispatch to OpenAI and Gemini deep-research agents. Because deep research is a **paid, non-deterministic** service, the template never calls it live in CI. Instead, `src/deep_research/deep_research_adapter.py` wires the real infrastructure request/result models (`DeepResearchConfig`, `DeepResearchRequest`, `DeepResearchResult`, `DeepResearchClient`) and ships a deterministic, offline path: `scripts/08_deep_research_dispatch.py` builds the genuine provider-neutral request and then *replays* a recorded report fixture (`src/deep_research/fixtures/recorded_report.json`), normalizing it through the real `DeepResearchResult` model. Replay fails closed if the fixture is missing — it never fabricates a passing run — mirroring the fixture-replay idiom of `template_sia`. The same adapter exposes `build_offline_request`, the exact call-site a live `submit` would dispatch, so a fork can enable real providers by supplying `OPENAI_API_KEY` / `GEMINI_API_KEY`:

Deep Research (Offline Fixture Replay) II

```
# Offline (default): replays the recorded report, no key r
uv run python scripts/08_deep_research_dispatch.py
```

Test Suite I

Every stage is covered by a no-mocks test suite (real computation and `pytest-httpserver` for network adapters) gated at $\geq 90\%$ statement coverage on `src/`. The suite covers:

- Record models and serialization (deduplication, canonical ID hierarchy)
- All 10 engine paths (arXiv, Semantic Scholar, OpenAlex, Crossref, PubMed, SovietRxiv, ChinaRxiv, Europe PMC, bioRxiv, medRxiv) with pytest-httpserver integration tests
- Search runner (multi-engine dispatch, relevance filtering, resume/clear, YAML config)
- Bibliometric analysis (subfield classification, temporal metrics, TF-IDF, NMF, citation network)
- Knowledge graph (schema, nanopublications, hypothesis scoring, LLM extraction)
- Visualization (headless figure generation, style config)
- Manuscript variable computation and injection